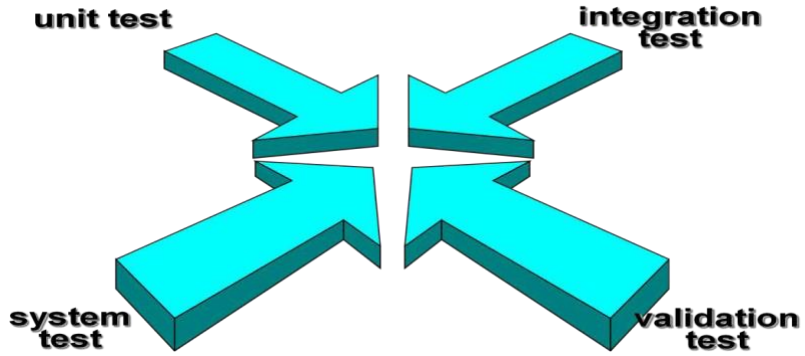
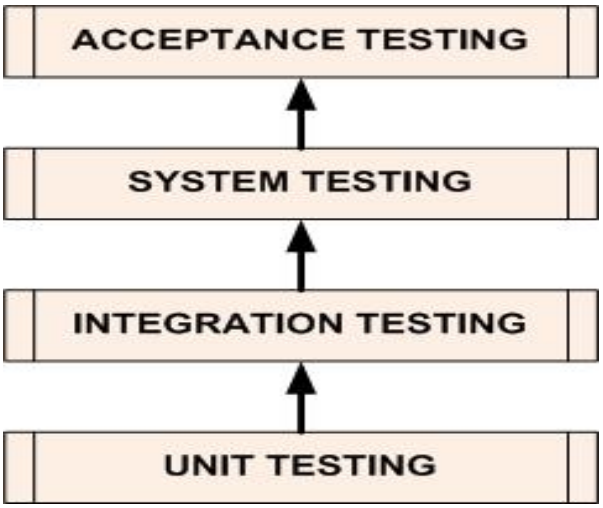
UNIT-3

## Test Strategies for Conventional Software:

**Testing Strategy:**



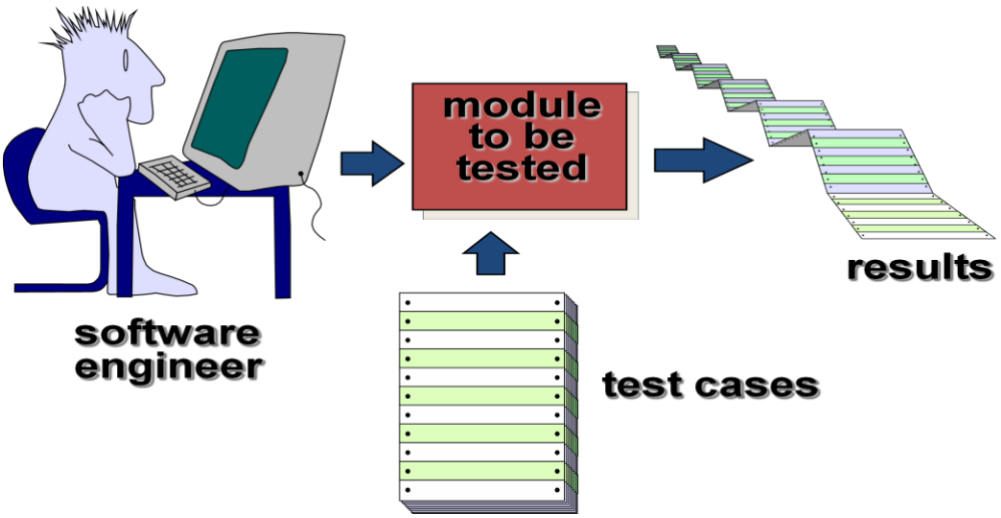




## Unit Testing:

- Unit Testing is defined as a type of software testing where individual components of

a software are tested. Unit Testing of software product is carried out during the development of an application.

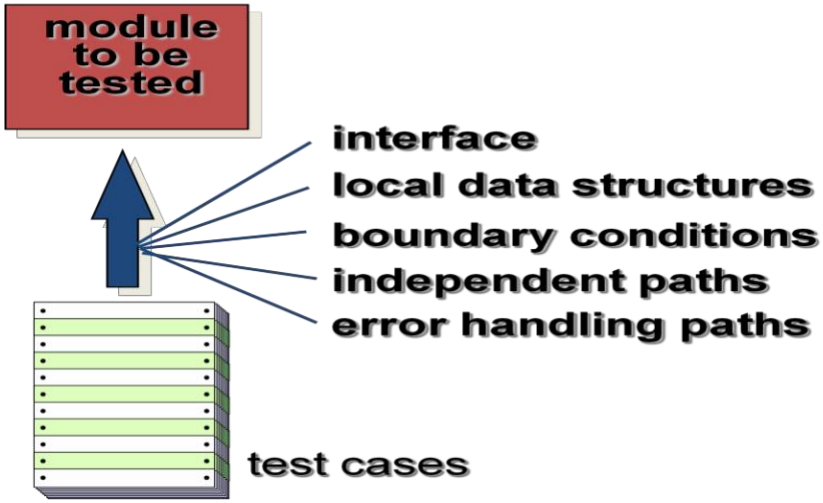
- Unit Testing is a level of software testing where individual units/ components of a software are tested.
- The purpose is to validate that each unit of the software performs as designed. A unit is the smallest testable part of any software. It usually has one or a few inputs and usually a single output.
- In procedural programming, a unit may be an individual program, function, procedure, etc. In object-oriented programming, the smallest unit is a method, which may belong to a base/ super class, abstract class or derived/ child class.
- Unit testing frameworks, drivers, stubs, and mock/ fake objects are used to assist in unit testing.



**Unit Testing Details:**

- Interfaces tested for proper information flow.
- Local data are examined to ensure that integrity is maintained.
- Boundary conditions are tested.
- Basis path testing should be used.
- All error handling paths should be tested.
- Drivers and/or stubs need to be developed to test incomplete software.

**Unit Testing:**




**Unit Test Environment:**

**Benefits of Unit Testing :**

- Unit testing increases confidence in changing/ maintaining code.
- Codes are more reusable. In order to make unit testing possible, codes need to be modular. This means that codes are easier to reuse.
- Development is faster.
- The cost of fixing a defect detected during unit testing is lesser in comparison to that of defects detected at higher levels. Compare the cost (time, effort, destruction, humiliation) of a defect detected during acceptance testing or when the software is live.
- Debugging is easy.
- Codes are more reliable.

**Integration Testing:**

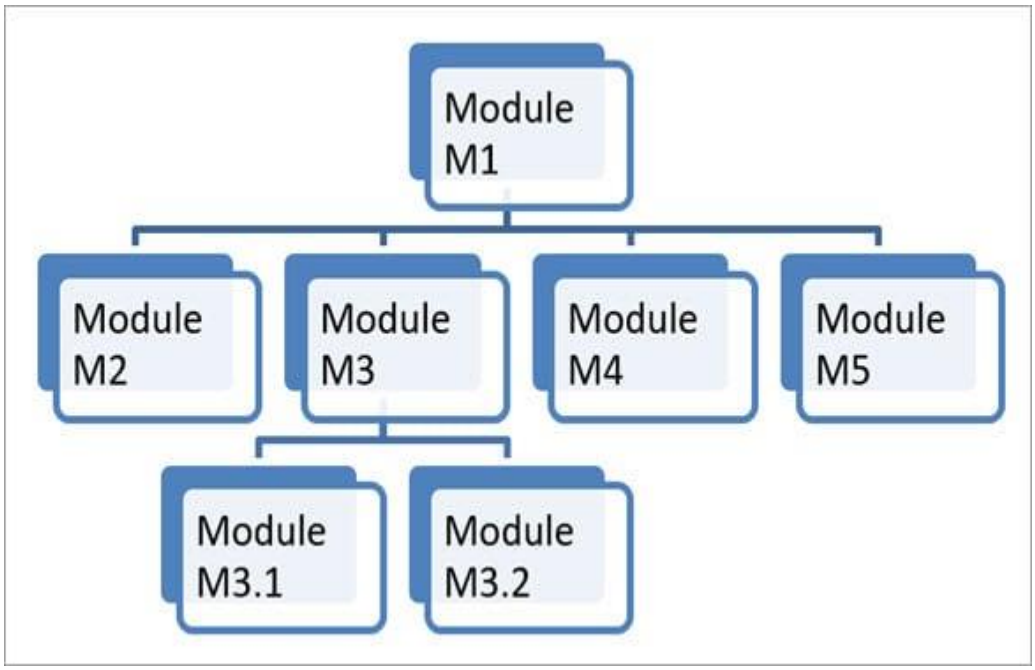
- Integration Testing, also known as Incremental Testing, is one of the approaches of Integration Testing and incorporates its fundamental concepts.

- In this testing, we test each module individually in unit testing phase, and then modules are integrated incrementally and tested to ensure smooth interface and interaction between modules.

- In this approach, every module is combined incrementally, i.e., one by one till all modules or components are added logically to make the required application, instead of integrating the whole system at once and then performing testing on the end product. Integrated modules are tested as a group to ensure successful integration and data flow between modules.

- As in integration testing, the primary focus of doing this testing is to check interface, integrated links, and flow of information between modules. This process is repeated till the modules are combined and tested successfully.

  Example
  System or software application consists of following Modules:

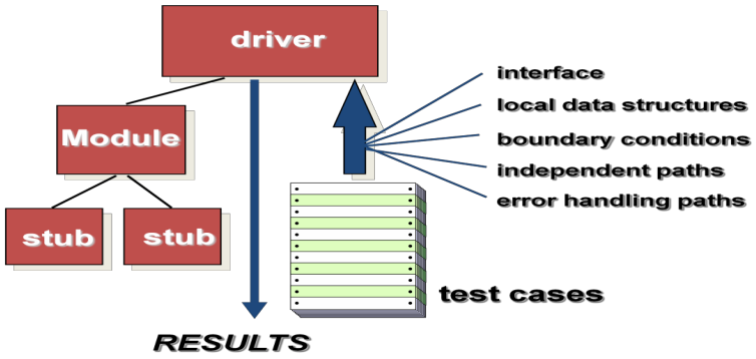


**Incremental Integration testing approach**

- Each Module i.e. M1, M2, M3, etc. are tested individually as part of unit testing
- Modules are combined incrementally i.e. one by one and tested for successful interaction
- In Fig2, Module M1 & Module M2 are combined and tested
- In Fig3, Module M3 is added and tested
- In Fig4, Module M4 is added and testing is done to make sure everything works together successfully
- Rest of the Modules are also added incrementally at each step and tested for successful integration

**Objective of Incremental Test :**

- To ensure that different modules work together successfully after integration
- Identify the defects earlier and in each phase. This gives developers an edge to identify where the problem is. Like if testing after M1 and M2 are integrated is successful but when M3 is added, the test fails; this will help the developer in segregating the issue
- Issues can be fixed in early phase without much rework and in less cost
  - Top - down testing (stubs).
  - Bottom - up testing (test harness).
    - Regression Testing.
    - Smoke Testing




**<u>Top Down Integration:</u>**

- As the name suggests, testing takes place from top to bottom, i.e., from the central module to sub module. Modules framing the top layer of application are tested first.

- This approach follows the structural flow of the application under testing. Unavailable or not developed modules or components are substituted by stubs.

- Main program used as a test driver and stubs are substitutes for components directly subordinate to it.

  - Subordinate stubs are replaced one at a time with real components (following the depth-first or breadth-first approach).

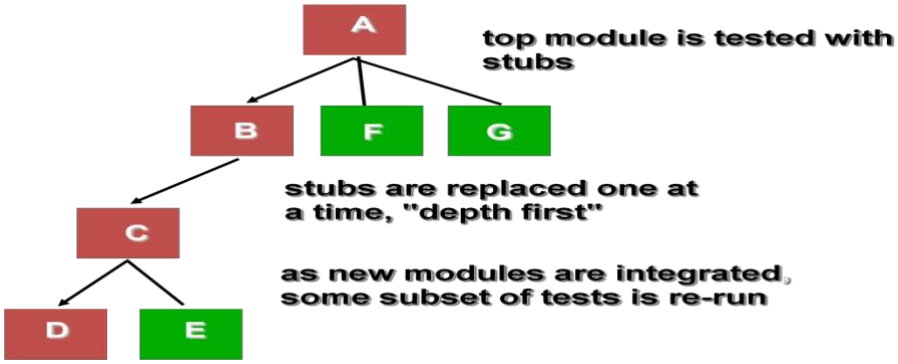
  - Tests are conducted as each component is integrated.

  - Set of tests and other stub is replaced with a real component.

  - Stubs and drivers are pseudo code or dummy code used in Integration or component

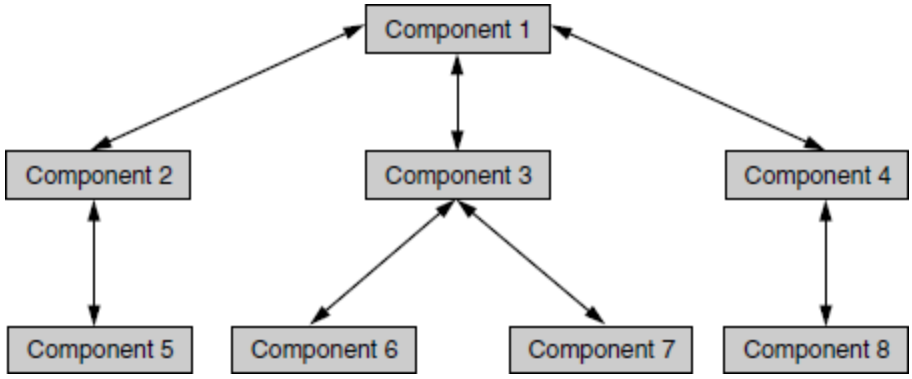
testing when one or more modules are not developed but are required to test some other module.

- **Stubs** are used in Top-down testing approach and are known as "called programs". Stubs help simulate the interface between lower lever modules which are not available or developed.




- Regression testing may be used to ensure that new errors not introduced.

- Example of Top-down integration:




Order of testing interfaces for the example is

**Step Interfaces tested**

1 1-2
2 1-3
3 1-4
4 1-2-5
5 1-3-6
6 1-3-6-(3-7)
7 (1-2-5)-(1-3-6-(3-7))
8 1-4-8
9 (1-2-5)-(1-3-6-(3-7))-(1-4-8)

In an incremental product development, where one or two components gets added to the product in each increment, the integration testing methodology pertains to only to those new interfaces that are added and those related interfaces that are impacted by the changes or increment in the product.

**Merits of Top-down Methodology**

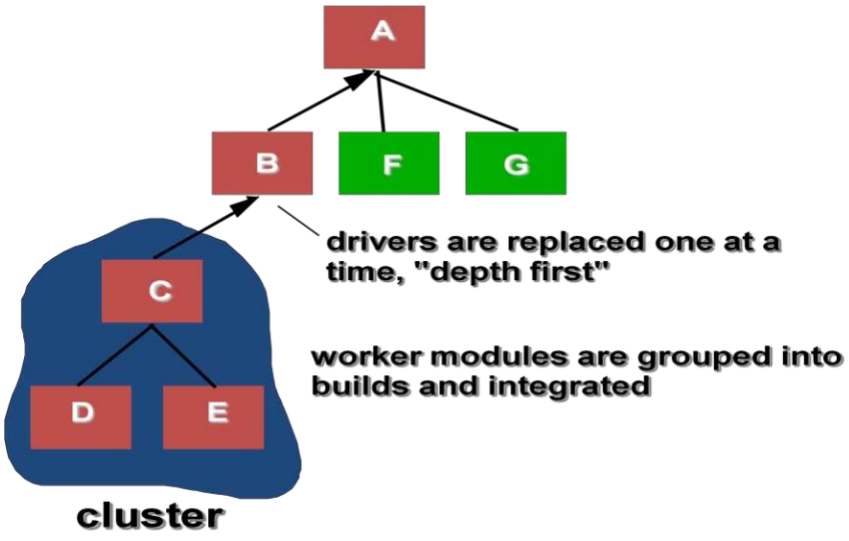
- Early exposure of architecture defects
- It outlines the working of an application as a whole in early stages and helps in early disclosure of design defects
- Main control points are tested early

**De-Merits of Top-down Methodology**

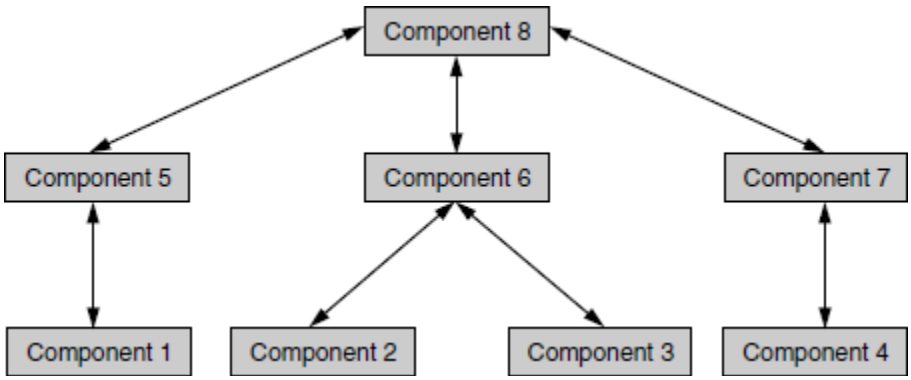
- Significant modules are tested late in cycle
- It is very challenging to write test conditions
- A stub is not a perfect implementation of related Module. It just simulates the data flow between two modules

**Bottom-Up Integration:**

- In this approach, testing takes place from bottom to top, i.e., modules at bottom layer are integrated and tested first and then sequentially other modules are integrated as we move up. Unavailable or not developed modules are replaced by drivers.
- Low level components are combined in clusters that perform a specific software function.
- A driver (control program) is written to coordinate test case input and output.
- The cluster is tested.
- Drivers are removed and clusters are combined moving upward in the program structure.



Example of bottom-up integration : Arrows pointing down depict logic flow; arrows pointing up indicate integration paths.




The navigation in bottom-up integration starts from component 1 in the above figure covering all sub-systems, till component 8 is reached. The order in which the interfaces have to be tested is depicted in the figure.

Order of interfaces tested using bottom up integration for

| **Step** | **Interfaces tested** |
|---|---|
| 1 | 1-5 |
| 2 | 2-6, 3-6 |
| 3 | 2-6-(3-6) |
| 4 | 4-7 |
| 5 | 1-5-8 |
| 6 | 2-6-(3-6)-8 |
| 7 | 4-7-8 |
| 8 | (1-5-8)-(2-6-(3-6)-8)-(4-7-8) |

Merits of Bottom-up Methodology

- This methodology is very useful for applications where bottom up design model is used
- It's easier to create test conditions in bottom up approach
- To start testing at the bottom level of hierarchy means testing of critical modules or functionality at an early stage. This helps in early discovery of errors
- Interface defects are detected at early stage

De-merits of Bottom-up Methodology

- Drivers are more difficult to write than stub
- Design defects are caught in the later stage
- In this approach, we do not have working application until the last module is build
- Driver is not a complete implementation of related Module. It just simulates the data flow between two modules,

**Step Interfaces tested**

**<u>Regression Testing:</u>**

- The selective retesting of a software system that has been modified to ensure that any bugs have been fixed and that no other previously working functions have failed as a result of the reparations and that newly added features have not created problems with previous versions of the software. Also referred to as verification testing, regression testing is initiated after a programmer has attempted to fix a recognized problem or has added source code to a program that may have inadvertently introduced errors. It is a quality control measure to ensure that the newly modified code still complies with its specified requirements and that unmodified code has not been affected by the maintenance activity.

**Regression Testing:**

- Regression test suit contains 3 different classes of test cases
  - Representative sample of existing test cases is used to exercise all software functions.
  - Additional test cases focusing software functions likely to be affected by the change.
  - Tests cases that focus on the changed software components.

**Smoke Testing:**

- Software components already translated into code are integrated into a build.
- A series of tests designed to expose errors that will keep the build from performing its functions are created.
- The build is integrated with the other builds and the entire product is smoke tested daily using either top-down or bottom integration.

**Validation Testing:**

- Ensure that each function or performance characteristic conforms to its specification.
- Deviations (deficiencies) must be negotiated with the customer to establish a means for resolving the errors.
- Configuration review or audit is used to ensure that all elements of the software configuration have been properly developed, cataloged, and documented to allow its support during its maintenance phase.

**Acceptance Testing:**

- Making sure the software works correctly for intended user in his or her normal work environment.
- Alpha test
  - version of the complete software is tested by customer under the supervision of the developer at the developer‘s site
- Beta test
  - version of the complete software is tested by customer at his or her own site

without the developer being present

**System testing**

System testing is defined as a testing phase conducted on the complete integrated system, to evaluate the system compliance with its specified requirements. It is done after unit, component, and integration testing phases.

The testing conducted on the complete integrated products and solutions to evaluate system compliance with specified requirements on functional and nonfunctional aspects is called system testing.

A system is a complete set of integrated components that together deliver product functionality and features. A system can also be defined as a set of hardware, software, and other parts that together provide product features and solutions. In order to test the entire system, it is necessary to understand the product's behavior as a whole. System testing helps in uncovering the defects that may not be directly attributable to a module or an interface. System testing brings out issues that are fundamental to design, architecture, and code of the whole product.

System testing is the only phase of testing which tests the both functional and non-functional aspects of the product. On the functional side, system testing focuses on real-life customer usage of the product and solutions. System testing simulates customer deployments. For a general-purpose product, system testing also means testing it for different business verticals and applicable domains such as insurance, banking, asset management, and so on.

**System Testing includes :**

1. **Performance/Load testing** :

    - To evaluate the time taken or response time of the system to perform its required functions in comparison with different versions of same product(s) or a different competitive product(s) is called performance testing.

    - Checks the run-time performance of software. Performance testing can be applied to understand your application or WWW site's scalability, or to benchmark the performance in an environment of third party products such as servers and middleware for potential purchase. This sort of testing is particularly useful to identify performance bottlenecks in high use applications. Performance testing generally involves an automated test suite as this allows easy simulation of a variety of normal, peak, and exceptional load conditions.

2. **Scalability testing** A testing that requires enormous amount of resource to find out the maximum capability of the system parameters is called scalability testing. This type of testing is explained in subsequent sections of this chapter.
3. **Reliability testing** To evaluate the ability of the system or an independent component of the system to perform its required functions repeatedly for a specified period of time is called reliability testing. This is explained in subsequent sections of this chapter.
4. **Stress testing** Evaluating a system beyond the limits of the specified requirements or system resources (such as disk space, memory, processor utilization) to ensure the system does not break down unexpectedly is called stress testing. This is explained in subsequent sections of this chapter.

5. **Interoperability testing** This testing is done to ensure that two or more products can exchange information, use the information, and work closely. This is explained in subsequent sections of this chapter.
6. **Localization testing** Testing conducted to verify that the localized product works in different languages is called localization testing.

**7. Compatibility testing**

This is used to ensure compatibility of an application or Web site with different browsers, OSs, and hardware platforms. Compatibility testing can be performed manually or can be driven by an automated functional or regression test suite.

**8. Load testing**

Load testing is a generic term covering Performance Testing and Stress Testing.

9. **Recovery testing**

– checks system‘s ability to recover from failures

10. **Security testing**

– verifies that system protection mechanism prevents improper penetration or data alteration.

**11. End-to-end testing**

Similar to system testing, involves testing of a complete application environment in a situation that mimics real-world use, such as interacting with a database, using network communications, or interacting with other hardware, applications, or systems if appropriate.

**12. Sanity testing**

Testing is to determine if a new software version is performing well enough to accept it for a major testing effort. If application is crashing for initial use then system is not stable enough for further testing and build or application is assigned to fix.

**13. Configuration Testing**

**14. Usability testing**

**Acceptance Testing:**

- Making sure the software works correctly for intended user in his or her normal work environment.
- Alpha test

version of the complete software is tested by customer under the supervision of the developer at the developer‘s site. In house virtual user environment can be created for this type of testing. Testing is done at the end of development. Still minor design changes may be made as a result of such testing.

- Beta test

version of the complete software is tested by customer at his or her own site without the developer being present. Testing is typically done by end-users or others. This is the final testing before releasing the application to commercial purpose.

–

.

**Verification vs validation:**

- Software testing is one element of a broader topic that is often referred to as verification and validation (V&V).
- Verification refers to the set of tasks that ensure that software correctly implements a specific function.
- Validation refers to a different set of tasks that ensure that the software that has been built is traceable to customer requirements.
- Boehm [Boe81] states this another way:
- Verification: "Are we building the product right?" The software should conform to its specification.
- Validation: "Are we building the right product?" The software should do what the user really requires.

**The V & V process:**

- As a whole life-cycle process - V & V must be applied at each stage in the software process.
- Has two principal objectives
  - The discovery of defects in a system
  - The assessment of whether or not the system is usable in an operational situation.

Strategic Testing Issues – 1 :

- Specify product requirements in a quantifiable manner before testing starts.
- Specify testing objectives explicitly.
- Identify the user classes of the software and develop a profile for each.
- Develop a test plan that emphasizes rapid cycle testing.

Strategic Testing Issues – 2:

- Build robust software that is designed to test itself (e.g. use anti-bugging).
- Use effective formal reviews as a filter prior to testing.
  - Conduct formal technical reviews to assess the test strategy and test cases.

| Verification | Validation |
|---|---|
| Are we implementing the system right? | Are we implementing the right system? |
| Evaluating products of a development phase | Evaluating products at the closing of the development process |
| The objective is making sure the product is as per the requirements and design specifications | The objective is making sure that the product meets user's requirements |
| Activities included: reviews, meetings, and inspections | Activities included: black box testing, white box testing, and grey box testing |
| Verifies that outputs are according to inputs or not | Validates that the users accept the software or not |
| Items evaluated: plans, requirement specifications, design specifications, code, and test cases | Items evaluated: actual product or software under test |
| Manual checking of the documents and files | Checking the developed products using the documents and files |

- As a whole life-cycle process - V & V must be applied at each stage in the software process.
- Has two principal objectives
  - The discovery of defects in a system
  - The assessment of whether or not the system is usable in an operational situation.
- Strategic Testing Issues - 1 Specify product requirements in a quantifiable manner before testing starts.
- Specify testing objectives explicitly.
- Identify the user classes of the software and develop a profile for each.
- Develop a test plan that emphasizes rapid cycle testing.

**Strategic Testing Issues – 2:**

- Build robust software that is designed to test itself (e.g. use anti-bugging).
- Use effective formal reviews as a filter prior to testing.
- Conduct formal technical reviews to assess the test strategy and test cases.

**Non-Functional Testing :**

Non-Functional Testing is defined as a type of Software testing to check non-functional aspects (performance, usability, reliability, etc) of a software application. It is designed to test the readiness of a system as per nonfunctional parameters which are never addressed by functional testin

**Functional testing versus non-functional testing.**

| Testing aspects | Functional testing | Non-functional testing |
|---|---|---|
| Involves | Product features and functionality | Quality factors |
| Tests | Product behavior | Behavior and experience |
| Result conclusion | Simple steps written to check expected results | Huge data collected and analyzed |
| Results varies due to | Product implementation | Product implementation, resources, and configurations |
| Testing focus | Defect detection | Qualification of product |
| Knowledge required | Product and domain | Product, domain, design, architecture, statistical skills |
| Failures normally due to | Code | Architecture, design, and code |
| Testing phase | Unit, component, integration, system | System |
| Test case repeatability | Repeated many times | Repeated only in case of failures and for different configurations |
| Configuration | One-time setup for a set of test cases | Configuration changes for each test case |

**Typical entry/exit criteria for non-functional tests.**

| Type of test | Parameters | Sample entry criteria | Sample exit criteria |
|---|---|---|---|
| Scalability | Maximum limits | Product should scale up to one million records or 1000 users | Product should scale up to 10 million records or 5000 users |
| Performance test | • Response time<br>• Throughput<br>• Latency | Query for 1000 records should have a response time less than 3 seconds | Query for 10,000 records should have response time less than 3 seconds |
| Reliability | • Failures per iteration<br>• Failures per test duration | There should be less than 2% failures when queries are run on 1000 records for 24 hours | There should be less than 0.1% failures when queries are run on 1000 records for 48 hours |
| Stress | System when stressed beyond the limits | Product should be able to withstand 25 clients login happening simultaneously for 5 hours in a configuration that can take only 20 clients | Product should be able to withstand 100 clients login simultaneously for 5 hours in a configuration that can take only 100 clients |